Programs for a Frozen Machine: The Description Length of Language and Task in Optimized Prefixes

Seminar report, Mechanistically Interpreting Multilingual Language Models (WT 2025/26)

Georgii Sirotenko

November 2026

Abstract

A frozen language model can be made to do a new job by placing a few trained vectors in front of its input. If the model is a fixed machine, those vectors are a program for it, and the length of the shortest program that makes the model do a job is a measure of how hard that job is for that model. This report measures that length for a frozen Qwen2.5-1.5B over three tasks (translation from English, factual recall, sentence continuation) and three output languages (Russian, German, English). Two lengths are used: the number of vectors, and the number of bits needed to specify them, measured with a variational code. Four findings. The number of vectors is not a useful length: translation needs one vector and recall needs many, and one trained vector beats a written instruction everywhere. In bits, telling the model to translate costs about 500 bits, telling it to recall facts in Russian costs about 2,400, and English recall is the most expensive job of all. Splitting a program into a shared task part and a language part does not save bits in either direction: a language given a task costs the same as a task given a language, and sharing a part between cells costs more than coding each cell alone. The difference between tasks is in how the model reads the program, not in its size: translation programs are read in the first layer, recall programs deep in the network. Along the way, the standard optimiser turned out to be the main obstacle: a smaller step size and the right checkpoint rule cut the measured lengths by 20–71%.

1 The question

Prompt tuning trains a short sequence of continuous vectors, placed before the input, while the model's own weights stay frozen (Lester et al. 2021; Li and Liang 2021). The engineering view is that this is a cheap way to adapt a model. This report takes a different view. A frozen model is a fixed machine. A prefix that makes it translate is a program for that machine, and the shortest such program is a description length of the task relative to the model: the number of bits one would have to send to a receiver who already has the model, so that the receiver can perform the task (Hinton and van Camp 1993; Blier and Ollivier 2018; Shaw et al. 2026).

Description lengths can be compared. For a multilingual model, the comparisons worth making are these.

- How long is the program for a task, and how does it change with the output language?
- Does the program split into a task part and a language part? If so, which part is the expensive one, and can the parts be shared between jobs or combined into new ones?
- How far apart are the programs for different jobs?
- Where in the network is the program read?

All lengths are relative to one model, one tokenizer and one input format. The results are orderings, differences and ratios within Qwen2.5-1.5B, not absolute facts about tasks or languages.

2 What was measured

2.1 The grid

Three tasks are crossed with three output languages. Translation is from English, so translation into English is not a task, and the grid has eight entries. Each entry is called a **cell** and written as task-language, for example recall-ru.

	English	Russian	German
translation from English	–	translation-ru	translation-de
factual recall	recall-en	recall-ru	recall-de
sentence continuation	continuation-en	continuation-ru	continuation-de

Every measurement is taken per cell. Translation and continuation use FLORES+ sentences; recall uses mLAMA (Kassner et al. 2021), a set of fill-in-the-blank facts such as "The capital of France is ___", asked in English with the answer expected in the target language.

2.2 A prefix program

A **prefix** is a sequence of k free vectors, each of the model's embedding size $d = 1{,}536$, placed in front of the input tokens. The model attends to them exactly as it would to token embeddings, but they are not tokens: they are trained by gradient descent, with every model weight frozen, to lower the model's loss on the target text. One such vector is called a **slot**. The whole project rests on this object: the prefix is the program, and the frozen model is the machine that runs it.

2.3 How good a program is: distortion

The model's loss on the target is measured in bits per target token, the standard teacher-forced cross-entropy. Call it L. Two reference values anchor each cell:

- L_{none}, the loss with no prefix at all: the model left to its own devices.
- L_{floor}, the best loss we know how to reach on that cell. It is the smaller of two numbers: the loss after fine-tuning the model's attention weights with LoRA (rank 16, which moves about 0.6% of the weights), and the loss of the best unpenalised prefix found. On five of the eight cells the prefix wins; on the three recall cells LoRA wins by a wide margin.

The **distortion** of a program is where its loss falls between these two:

$$\hat{D} = \frac{L - L_{\text{floor}}}{L_{\text{none}} - L_{\text{floor}}}.$$

$\hat{D} = 1$ means the program is useless and $\hat{D} = 0$ means it is as good as the floor. A worked example: on translation-ru the no-prefix loss is 1.92 bits per token and the floor is 1.34. A program that brings the loss to 1.38 has closed 94% of that gap, so its distortion is 0.06.

Programs of different cost trade cost against quality, so costs are compared at fixed quality. An **operating point** is a distortion threshold at which the cost is read off. Two were fixed before any test number was seen: $\hat{D} \leq 0.25$ (the program closes three quarters of the gap) and $\hat{D} \leq 0.10$ (nine tenths). A third, $\hat{D} \leq 0.05$, is reported as exploratory.

Continuation turns out to be unmeasurable. Its floor lies only 0.10–0.16 bits per token below its no-prefix loss, which is less than the variation between training seeds, so the denominator of $\hat{D}$ is noise. Continuation is kept in the tables but excluded from every claim. The five measurable cells are translation-ru, translation-de, recall-ru, recall-de and recall-en.

2.4 How long a program is: two units

Slots. The first unit is simply k, the number of vectors. Sweeping k with free, unpenalised vectors gives a curve of distortion against k; the **slot complexity** k^* of a cell is the smallest k that reaches an operating point.

Bits. The second unit is a description length. A single vector of 1,536 real numbers could carry any amount of information, so counting vectors says nothing about how much information the program holds. The variational code (Hinton and van Camp 1993) measures it. Each slot is no longer a fixed vector but a Gaussian distribution $q_i = \mathcal{N}(\mu_i, \operatorname{diag}\sigma_i^2)$ over vectors, with μ_i and σ_i trained. At every forward pass one vector is drawn from each slot's distribution, and the model runs on the drawn prefix (the figure below). The cost of a slot is how far its distribution departs from a reference distribution, the **prior** π_0, which is a Gaussian fitted to the model's own token-embedding matrix: a slot that looks like a typical token embedding is free, and the bits pay for the displacement from that. The cost in bits is $\mathrm{KL}(q_i \,\|\, \pi_0)/\ln 2$, and the cost of the prefix is the sum over slots. By the bits-back argument this is exactly the number of bits a sender needs to transmit a sample from q to a receiver who knows the prior; the receiver ends up holding one drawn prefix, which is why distortion is measured on draws and not on the mean μ.

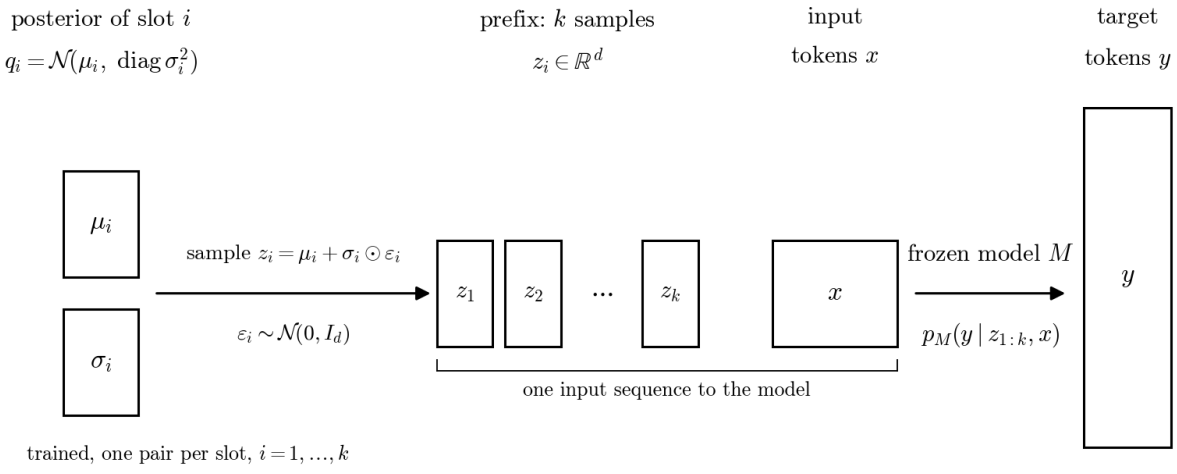


Figure 1: The sampling step of a variational prefix. Each slot i is a diagonal Gaussian posterior $q_i = \mathcal{N}(\mu_i, \operatorname{diag}\sigma_i^2)$ with trained μ_i and σ_i; d is the model's embedding dimension (1,536), so a slot is a vector of the same size as a token embedding. At every forward call one sample $z_i = \mu_i + \sigma_i \odot \varepsilon_i$ is drawn per slot, the k samples form the prefix, and the prefix followed by the input tokens x is one sequence for the frozen model M, whose score of the target tokens y is the distortion. The samples are the coded object; the rate of slot i is $\mathrm{KL}(q_i \,\|\, \pi_0)/\ln 2$ bits against the prior π_0 fitted to the token-embedding matrix, and the prefix's bits are the sum over slots.

Training minimises the loss plus β times the bits per training token. A large β buys a short program at higher loss; a small β the reverse. Running several k and β gives a cloud of (bits, distortion) points per cell. The **frontier** of a cell at an operating point is the smallest bit count among the runs whose distortion is under the threshold. The headline numbers of this report are frontier values.

2.5 Splitting a program into a task part and a language part

Four further measurements ask whether the program has parts.

Hierarchy. A group of cells (the languages of one task, or the tasks of one language) is coded jointly as a shared **centre** c plus one **deviation** δ_m per member; the member's prefix is $c + \delta_m$. The centre is priced against the embedding prior like any prefix and paid once for the group. A deviation is priced against a Gaussian around zero with the same width as the prior, so that moving a deviation by some amount costs

the same bits as moving a whole prefix by that amount. Grouping by task makes the centre the task part and the deviations the language parts; grouping by language reverses the roles. This gives task bits and language bits in one currency, and two questions: which **nesting** is cheaper, and is the group's total (centre plus deviations) smaller than coding each cell alone?

Composition. One tied task part per task and one tied language part per language, concatenated into a prefix. Trained on all cells, or with some cells held out and assembled for them without training.

Distance and transfer. The cost of training from one cell's prefix toward another, relative to training from scratch, and the distortion of a prefix applied unchanged to another cell.

Mechanism. For a trained prefix, the hidden states at the prefix positions are copied into the attention input of one transformer block of a run with a random prefix. The share of the trained prefix's effect that this single block recovers says how much of the program is read there.

3 Setup

Model and data. Qwen2.5-1.5B base, frozen, in bfloat16 on one A100. FLORES+ dev sentences for training and model selection, devtest for every reported number; mLAMA facts split by item so that no fact appears in both training and test. Test sets have 1,012 sentences per translation cell and 4,451–6,496 facts per recall cell.

Runs. Slot curves: $k \in \{1, 2, 4, 8, 16, 32\}$, two seeds, 1500 steps of AdamW, batch 16, no penalty; repeated with the target language written before the target, with the task written as an instruction, and with a full natural-language prompt and no prefix. LoRA floor per cell. Variational prefixes: $k \in \{2, 4\}$, $\beta \in \{1, 3\}$, two seeds, on the five measurable cells. Hierarchies: both nestings on the same (k, β) grid, one seed. Composition: $1{+}1$, $2{+}2$ and $4{+}4$ slots with five hold-out sets. Distance: every cell's $k = 4$ prefix trained toward every cell over 512 examples. Mechanism: eight cells, five probes, all 28 blocks. In all about 1,900 result rows and 36 A100 hours. Every number and figure in this report is regenerated from the result tables by one notebook.

Two training protocols. The variational runs were first done under a preregistered protocol (learning rate 0.05, checkpoint chosen by the loss of the mean prefix) and then, after the training histories showed that this protocol inflates the bit counts, under a corrected one (learning rate 0.01, checkpoint chosen by loss plus β times bits, both scored on draws). The corrected runs are the headline throughout. The preregistered runs, and what went wrong with them, are the subject of the section on optimisation; they are the reason the hierarchy comparisons are reported like for like under one protocol.

4 Results

4.1 Translation needs one slot; slots are not a length

The slot curves separate the tasks at once. Translation reaches $\hat{D} \leq 0.10$ with a single slot in both languages and does not improve with more. Recall keeps improving all the way to 32 slots in Russian and German, and the training budget is binding: a 3000-step run on recall-ru at $k = 8$ lowers $\hat{D}$ from 0.043 to 0.032 and is still falling. Continuation is flat noise, as explained above.

cell	$k^*(0.25)$	$k^*(0.10)$	$k^*(0.05)$, exploratory
translation-ru	1	1	1
translation-de	1	1	2
recall-ru	1	2	4
recall-de	1	4	32
recall-en	2	16	not reached

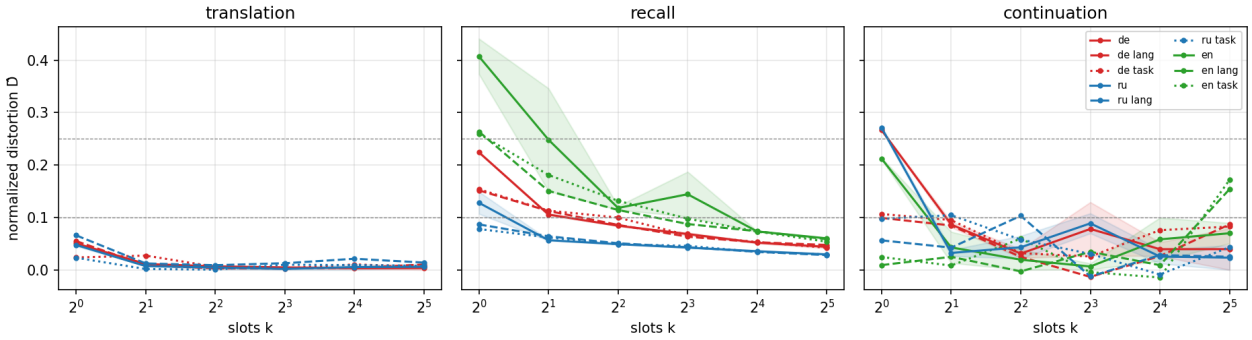


Figure 2: Distortion against the number of slots, unpenalised prefixes, mean over two seeds with the seed range shaded. Dashed lines mark the operating points. Line style is the conditioning: nothing given, target language given, task given.

Table 1. Slot complexity: the smallest number of slots reaching each operating point. Seed differences are below 0.011 in $\hat{D}$ on translation and 0.042 on Russian and German recall, but 0.20 on English recall, the one cell where the slot curve is unreliable.

Two observations show why the number of slots cannot serve as a description length.

First, one trained slot carries far more than an instruction. A written prompt that states the task and the language, scored with no prefix, leaves recall at $\hat{D}$ 0.91–0.96 and translation at 0.12–0.20. One trained slot brings recall to 0.13–0.41 and translation to 0.05. If a slot were worth about one token, the instruction, which is many tokens, would win.

Second, the slot count cannot see what the model is told. Writing the target language or the task into the input changes $k^*(0.10)$ in none of the translation cells and in two recall cells (recall-ru from 2 to 1, recall-en from 16 to 8), and it does so identically whether the language or the task is given. Averaged over the measurable cells, either piece of information saves 1.8 slots, and 8 of those 9 saved slots come from recall-en alone. In slots, telling the model the language and telling it the task look the same. In bits they do not, as the next sections show.

4.2 In bits: 500 for translation, 2,400 and more for recall

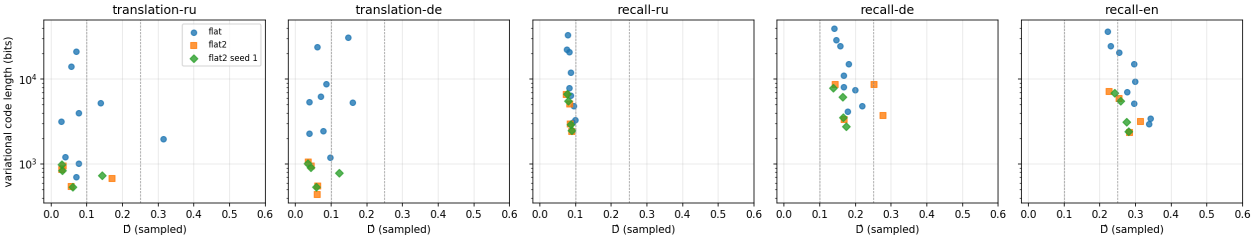


Figure 3: Bits against distortion for every variational run, per cell. Squares are the corrected protocol, circles the preregistered one. Distortion is scored on draws from the trained distribution. The frontier of a cell is the lowest square under an operating point.

Table 2 is the central result: the fewest bits that reach each operating point, under the corrected protocol.

cell	bits at $\hat{D} \leq 0.10$	bits at $\hat{D} \leq 0.25$	best $\hat{D}$ reached
translation-de	436	436	0.036

cell	bits at $\hat{D} \leq 0.10$	bits at $\hat{D} \leq 0.25$	best $\hat{D}$ reached
translation-ru	542	542	0.030
recall-ru	2,419	2,419	0.075
recall-de	not reached	3,343	0.143
recall-en	not reached	7,155	0.226

Table 2. Description length in bits, corrected protocol, first seed. For translation and Russian recall the cheapest run under 0.25 is already under 0.10, so the two columns coincide. A second seed reproduces these values within 4% on translation-ru, recall-ru and recall-en and within 22% on the other two cells, in opposite directions, with the same ordering of cells (appendix).

What the numbers say. Telling this model to translate English into German or Russian costs about 500 bits, which is less than a tweet. Telling it to recall facts in Russian costs 2,400 bits, five times as much, and the program still leaves 7–9% of the gap unclosed. German and English recall do not reach 10% excess loss within the training budget at all and cost 3,300 and 7,200 bits at the looser 25% level.

The ordering across tasks follows how much there is to learn: the gap between no-prefix loss and floor is 3.5–6.2 bits per token on recall and only 0.6–0.9 on translation, because the base model already translates somewhat and barely recalls facts in the target languages at all.

The ordering across languages does not follow how much the model has seen of each language. English recall is the dearest program, not the cheapest, in slots (16 at the 0.10 level) and in bits (7,155 at 0.25). This survives restricting the three recall test sets to the 4,212 facts they share ($\hat{D}$ at $k = 4$: Russian 0.049, German 0.079, English 0.122), and the English LoRA floor is also the worst of the three (2.58 bits per token against 1.10 for Russian). What is expensive in English is not the language but the particular facts that mLAMA asks about in English, which this model does not know. The expected language ordering (preregistered hypothesis H1) is not supported.

4.3 Splitting into task and language saves nothing

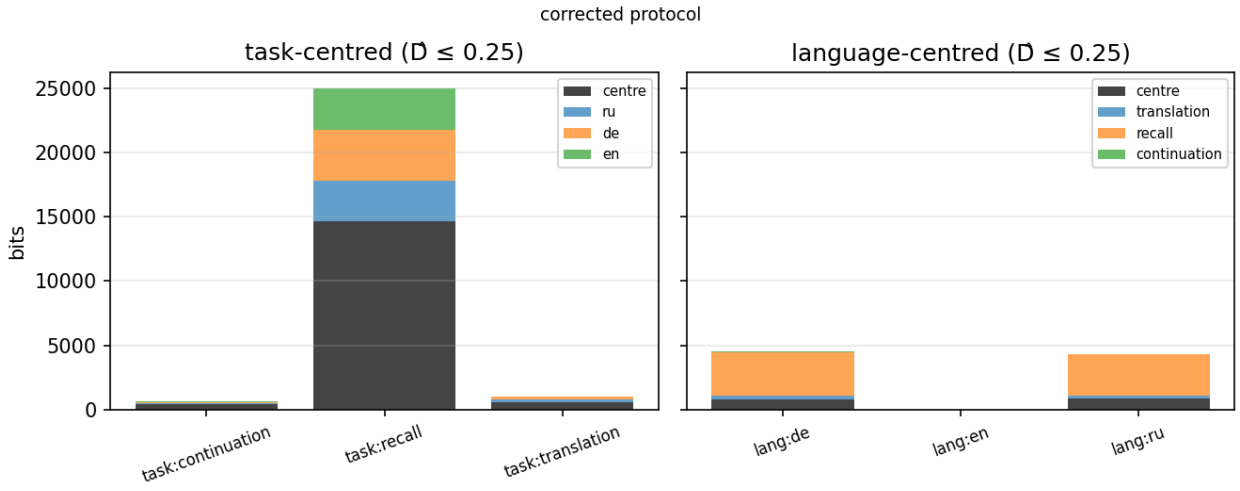


Figure 4: Bits at $\hat{D} \leq 0.25$ for the two nestings under the corrected protocol: a group's cost is its centre, paid once, plus one deviation per member.

The hierarchy codes a group of cells as one centre plus one deviation per member, and a group only counts as reaching an operating point when every member reaches it in the same run. Table 3 compares each group's cost with the cost of coding its cells independently, that is, the sum of the Table 2 values over the group.

group ($\hat{D} \leq 0.25$)	hierarchy, bits (k, β)	independent, bits	ratio
translation: ru, de	1,029 (4, 3)	978	1.05
recall: ru, de, en	24,972 (4, 1)	12,917	1.93
continuation: ru, de, en (unmeasurable)	667 (4, 3)	3,724	0.18
Russian: translation, recall, continuation	4,353 (2, 3)	4,204	1.04
German: translation, recall, continuation	4,560 (2, 3)	4,227	1.08
English: recall, continuation	not reached	9,188	–

Table 3. Grouping by task (top three rows) and by language (bottom three) against independent programs, corrected protocol. The independent continuation values come from the preregistered runs, the only variational runs on continuation. Over all eight cells the task-centred code costs 26,700 bits against 17,600 for independent programs; the language-centred code never gets English recall under 0.25 (best 0.28) and so does not code the grid.

Sharing does not compress. The task-centred hierarchy costs 1.5 times the independent programs over the grid: parity on translation and 1.9 times on recall. The language-centred one is at parity on Russian and German and fails on English. The recall factor decomposes into two parts. At the same (k, β) a hierarchy costs 0.94–1.5 times the sum of the flat runs with the same hyper-parameters, at the same or slightly worse distortion of its worst member; the shared centre buys nothing. The frontier comparison then adds the cost of one β serving every member: on recall the only task-centred run in which all three languages reach 0.25 is $(k, \beta) = (4, 1)$, whose centre alone (14,600 bits) costs more than the three independent programs together. The factor 1.93 is 1.12 (the excess at that grid point, 24,972 against 22,360) times 1.73 (one β for all members, 22,360 against 12,917).

Neither nesting is cheaper. At matched (k, β) the language-centred code costs 0.88–1.24 times the task-centred one over all eight cells (Table 4), at worse worst-member distortion in three of the four grid points and better in the fourth.

(k, β)	task-centred, bits (worst $\hat{D}$)	language-centred, bits (worst $\hat{D}$)	ratio
(2, 1)	24,591 (0.28)	30,448 (0.59)	1.24
(2, 3)	12,693 (0.56)	11,182 (0.48)	0.88
(4, 1)	28,659 (0.23)	30,532 (0.28)	1.07
(4, 3)	11,180 (0.28)	13,396 (0.37)	1.20

Table 4. The two nestings at the same hyper-parameters: three centres plus eight deviations either way, summed over all eight cells.

A language given a task costs what a task given a language costs. Given the translation centre (604 bits), a target language costs 225 bits (Russian) or 200 (German), less than half of a standalone translation program. Given a language centre (830–900 bits), translation costs 201 (Russian) or 257 (German). Given the recall centre (14,600 bits), a language costs 3,200–3,950 bits, more than a standalone Russian or German recall program; given a language centre, recall costs 3,200 (Russian) or 3,400 (German), and for English no run gets under 0.25. Continuation given a language centre costs 60 bits. Recall is the part that cannot be shared with anything: it costs about 3,200 bits as a deviation from either kind of centre, and whichever group has to carry it is the expensive group.

For translation alone the picture is simple. The centre costs more than either specialised program (604 against 542 and 436 bits) and each deviation less than half of one, so centre plus deviation cannot beat two independent programs: 1,029 against 978 bits. The preregistered hypothesis H2, that one of the two parts is the cheap modifier of the other, fails in both directions. The task given the language costs 0.9–1.0 times the language given the task on recall and 0.9–1.3 times on translation, which agrees with the slot axis, where the two savings were equal too.

4.4 Composition: parts combine for translation, not for recall

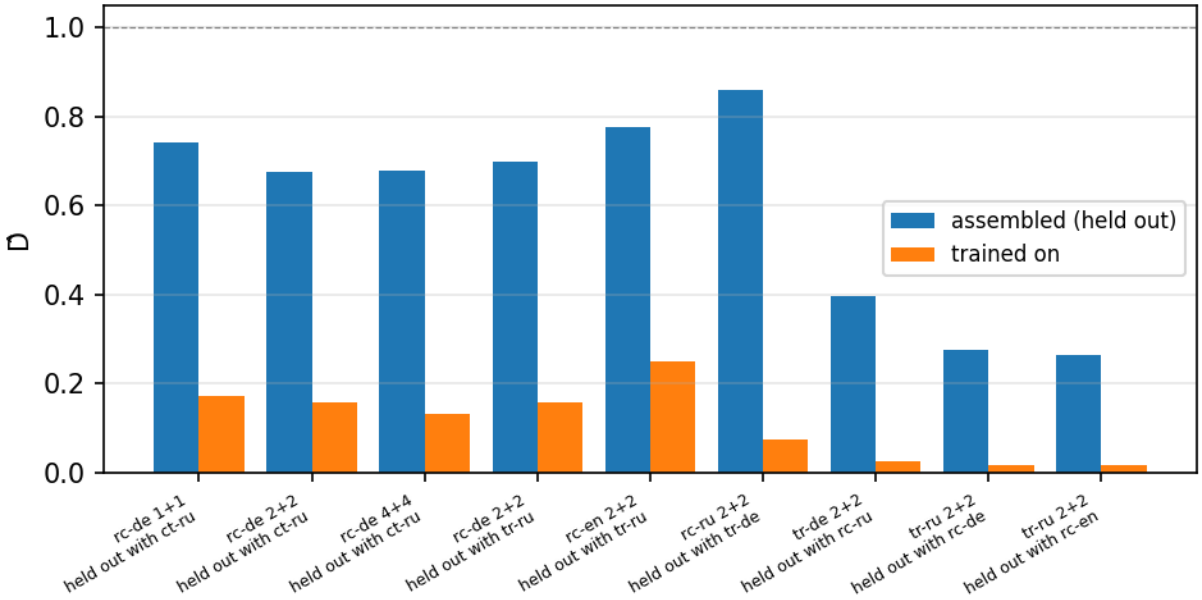


Figure 5: Distortion of tied task and language parts assembled for a held-out cell (vertical) against the same parts trained on that cell (horizontal).

Six tied parts (three task parts, three language parts) trained on every cell match free prefixes at the same total number of slots on translation ($\hat{D}$ 0.017–0.027 against 0.005–0.007 at $2{+}2$ slots) and cost about twice the distortion on recall (0.08–0.25 against 0.05–0.12). Assembled without training for a held-out cell, the parts close 60–73% of the gap between no prefix and the floor on translation (three hold-outs over two cells, $\hat{D}$ 0.27–0.40) and only 14–32% on recall (six hold-outs over three cells, $\hat{D}$ 0.68–0.86). Task and language parts compose partially for translation and barely for recall, which is the same picture the per-member costs gave. The resource-level part of hypothesis H3 (a Chinese comparison) was not run.

4.5 Programs do not transfer

Starting from a prefix of the same task costs 0.85 of training from scratch; starting from the same language or an unrelated cell costs 0.95. A clustering score (silhouette) of the task grouping on these distances is 0.004 and of the language grouping -0.09: the direction expected by hypothesis H4 but no structure. The matrix is one seed with seven pairs per group, and several warm starts cost more than scratch, so it measures optimisation noise as much as distance.

The strong signal is in the right panel. A prefix applied unchanged to any other cell is about as bad as no prefix or worse ($\hat{D}$ 0.7–1.4, median 0.97, up to 2.5), and a recall prefix applied to continuation is catastrophic ($\hat{D}$ 5–25). The recall program overrides the model's default behaviour rather than adding to it.

4.6 Where the program is read

Copying the prefix positions into all blocks reproduces the trained prefix exactly. Copying into block 0 alone recovers 70–87% of the effect for translation and 74–94% for continuation, but only 36–58% for recall, which has a second peak at block 17 and a smaller one at blocks 10–11. The single-block shares summed over all blocks come to 2.6 for translation and 3.0 for continuation, so their reading is redundant, but only 1.3 for



Figure 6: Left: cost of training from one cell's prefix toward another, divided by the cost from scratch, symmetrised. Right: distortion of a trained prefix applied unchanged to another cell.



Figure 7: Share of a prefix's effect recovered by copying its positions' hidden states into the attention input of one block, mean over cells per task. Undefined where the trained prefix is not at least 0.1 bits per token better than the random one, which excludes the deviation probes on continuation and on translation-ru.

recall, which is close to additive. A translation program is copied into the token stream in the first block and not needed afterwards. A recall program is read in several places, deep into the network.

The preregistered comparison (hypothesis H5) asked at which block the language deviation and the task centre of a hierarchical prefix are read most. The answer is block 0 for both, so the peak does not separate them. The profiles do. The task centre is read early: on recall its share summed over blocks 0–3 is 1.0 and over blocks 14–27 it is 0.5. The language deviation on top of it is read early and again deep: on recall 1.6 early and 1.9 late, with block 17 alone recovering 0.94 of its effect; on translation-de 1.2 early and 2.1 late, with blocks 16 and 22 recovering 0.99 and 0.85. A task deviation under a language centre behaves like the task centre. So there is a depth split by part, opposite in direction to what H5 expected: the task part is read early, the language part early and late. One detail: the recall centre without its English deviation is worse than a random prefix on English, so the recall centre is a non-English program. These probes were run on the preregistered hierarchies; on the corrected ones, probed at their means, the split replicates where the probe is defined, and a probe on draws is implemented but not yet run.

4.7 Compressing the input (exploratory)

With the $k = 4$ task prefix in place, one per-example vector replaces the whole input to within 10% of the real-input loss on every cell but translation-ru, which needs two; without the prefix, translation and continuation need two. There is no bit cost on these vectors, so this is a slot count. One oddity for the record: on recall the bare model scores the target better with no input (3.1 bits per token) than with the real query (7.2), consistent with confident wrong answers once a subject is named.

5 The objective is right, the optimiser is not

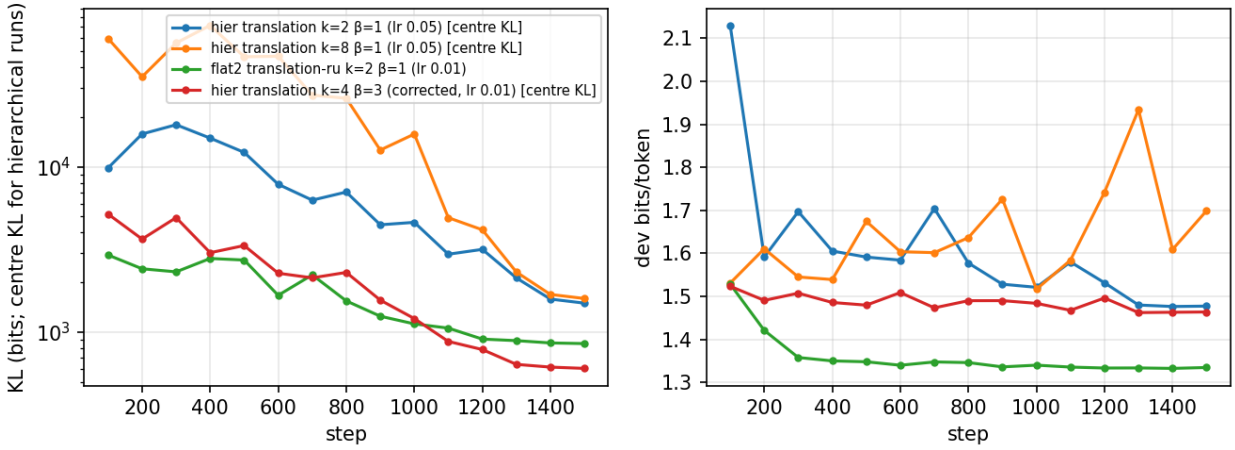


Figure 8: Bits and development loss against training step. Left two panels: hierarchical runs under the preregistered protocol. Right two: a flat run and a hierarchical run under the corrected protocol.

The variational objective rewards short programs, but the standard optimiser does not find them. The saved histories of the 27 preregistered hierarchical runs show three failures. With learning rate 0.05, every Adam step moves a slot by about two prior standard deviations, and the centre's bit count overshoots early, peaking between steps 100 and 800 at a median of 26,000 bits for $k = 2$ (range 6,000–182,000) and 144,000 for $k = 8$ (28,000–549,000), before the decaying learning rate lets the penalty pull it back. The checkpoint rule, which picked the step with the lowest loss of the mean prefix, often picked a step where the bit count was far above its final value: the selected count exceeded the final one by a median factor of 1.1–1.2 and up to 20. And the mean prefix is not what the code transmits: on translation at $\beta = 3$ the mean scores $\widehat{D}$ 0.35–0.56 in three of four runs while draws score 0.06–0.17, because the model was trained on draws.

The corrected protocol (learning rate 0.01, checkpoint chosen by loss plus β times bits, both scored on draws, and σ initialised at the prior) removes all three. In the 12 corrected hierarchical runs the centre's bits peak at step 150–200 at 3,600 ($k = 2$) to 5,700 bits ($k = 4$), 7–12 times lower, then fall monotonically, and the selected checkpoint is within 2% of the final count. The measured frontier drops on every cell: by 22% on translation-ru and recall-ru, 58–63% on translation-de, 20% on recall-de and 71% on recall-en, with four runs per cell instead of nine. All corrected bit counts are still falling at the end of the 1500-step budget, so they are upper bounds.

The correction also changed a result. Under the preregistered protocol the task-centred hierarchy coded all eight cells for 37% fewer bits than the language-centred one and 11% fewer than independent programs (33,500 against 53,500 and 37,600 bits). Both margins came from language-centred recall deviations of 5,300–26,200 bits, which the corrected protocol brings down to 3,200–3,400, and neither survives it. They were artefacts of the checkpoint rule, not properties of the model.

Shaw et al. (2026) report that on a parity task the same objective reaches 870 bits from a hand-built initialisation but 2.8 million bits from random initialisation, and leave the optimiser as the open problem. At this scale most of that gap closes by selecting on the objective and stepping at the scale of the prior.

6 What the results mean

The question was how the program for "do task t in language l" is organised. The answer, for this model, is that it is not organised as a cheap modifier on a shared core in either direction. The task fixes the size: translation is a few hundred bits whatever the language, recall a few thousand. The language part is small for translation (about 200 bits on top of the task) and large for recall (about 3,200 bits), but so is the task part given the language, and sharing a centre between cells never saves bits. What distinguishes the tasks is the mechanism. A translation program is read in the first block and transfers partially between languages; a recall program is read deep in the network, is language-specific, overrides the model's defaults, and cannot be shared, composed or transferred.

Two of the findings are robust under a second seed and both protocols: the task ordering in bits and the failure of sharing. The nesting comparison rests on four runs per group and one seed. The distance matrix is too noisy to carry a claim. The most general finding is the one about the optimiser, since it concerns the measurement itself rather than this model.

7 Preregistered hypotheses

	comparison	outcome
H1	cost ordering by language exposure and by task difficulty	task ordering supported (recall far above translation); language ordering reversed on recall (English dearest), robust to common items; exposure measure unavailable
H2	slots saved by giving the language vs the task; bits saved by the hierarchy	equal on the slot axis (1.8 slots each, 8 of 9 from one cell); equal in bits (task given language 0.9–1.0 times language given task on recall, 0.9–1.3 on translation); the preregistered 1.3–7.8 asymmetry on recall was an optimiser artefact

	comparison	outcome
H3	composition cost at $\widehat{D} = 0.25$; bits saved by a shared centre	translation composes partially (60–73% of the gap zero-shot), recall barely (14–32%); the shared centre saves no bits: 1.5 times independent programs (parity on translation, 1.9 on recall); nestings equal within 25%; Chinese not run
H4	clustering of distances by task vs by language	0.004 vs -0.09: task direction, no structure, not significant; same-task warm start 0.85 vs 0.95
H5	block of maximal recovery, language deviation vs task centre	block 0 for both; the language deviation is read again at blocks 16–17 and the task centre is not: a split by part, opposite to H5
H6	per-example input vectors needed, per sentence and per token	one vector suffices with the task prefix; per-token variant not computed; no bit cost

8 Deviations and limitations

Three deviations from the preregistration are recorded in the project ledger. The floor is the better of LoRA and the best prefix, because prefixes beat LoRA on five cells. Continuation is excluded from headline claims as unmeasurable. The variational protocol was corrected after the preregistered runs, and the corrected runs are the headline; both are reported.

Two seeds were run where the proposal planned three; the second seed of the corrected protocol is a replication of the frontier (appendix), and the hierarchies and the distance matrix are one seed. The recall budget binds at 1500 steps. The hierarchical comparisons are like for like but rest on four runs per group and one deviation prior width. Every corrected bit count is still falling at the end of training, so all are upper bounds. One corrected run (English centre, $k = 2$, $\beta = 1$) selected a transient dip in the development loss at step 500; selection on the objective is not immune to noise. The reported distortion of a variational prefix averages eight passes over the test set with a fresh draw per batch, which estimates the expected loss over draws well but says nothing about how much a single drawn program varies. The mechanism probes patch attention inputs at the prefix positions; a residual-stream patch would answer a slightly different question. An early version of the hierarchy priced deviations against a prior forty times wider than the embedding prior; those runs are kept as a control and not used.

9 What a research version would do

The same code runs the scale axis (Qwen2.5 0.5B to 7B), twenty or more FLORES languages stratified by resource, further tasks, and discrete prefixes. Three changes follow from this report. Select every variational checkpoint on the objective and score distortion on draws. Replace the learning rate by a step size in units of the prior width. Adopt the per-column Gaussian-mixture prior of Shaw et al. (2026) as a second bit count, and charge $\log_2$ of the number of k values tried for the choice of k. The hypotheses worth the compute are the ones this report could not settle: the exposure ordering across many languages, the composition cost against resource level, and whether the depth at which a program is read changes with scale.

10 References

- Blier, L. and Ollivier, Y. (2018). The Description Length of Deep Learning Models. NeurIPS.
- Dumas, C. et al. (2025). Separating Tongue from Thought. ACL.
- Hendel, R., Geva, M. and Globerson, A. (2023). In-Context Learning Creates Task Vectors. Findings of EMNLP.
- Hinton, G. E. and van Camp, D. (1993). Keeping the Neural Networks Simple by Minimizing the Description Length of the Weights. COLT.
- Kassner, N., Dufter, P. and Schütze, H. (2021). Multilingual LAMA. EACL.
- Khashabi, D. et al. (2022). Prompt Waywardness. NAACL.
- Lester, B., Al-Rfou, R. and Constant, N. (2021). The Power of Scale for Parameter-Efficient Prompt Tuning. EMNLP.
- Li, X. L. and Liang, P. (2021). Prefix-Tuning. ACL.
- Shaw, P., Cohan, J., Eisenstein, J. and Toutanova, K. (2026). Bridging Kolmogorov Complexity and Deep Learning. ICLR. arXiv:2509.22445.
- Todd, E. et al. (2024). Function Vectors in Large Language Models. ICLR.
- Voita, E. and Titov, I. (2020). Information-Theoretic Probing with Minimum Description Length. EMNLP.
- Vu, T. et al. (2022). SPoT: Better Frozen Model Adaptation through Soft Prompt Transfer. ACL.

11 Appendix: replication and reproduction

The second seed of the corrected protocol, for the frontier of Table 2. Each seed is an independent replicate of the whole run grid.

cell	seed 0, bits at 0.25 (k, β)	seed 1, bits at 0.25 (k, β)	ratio
translation-ru	542 (4, 3)	534 (4, 3)	0.99
translation-de	436 (2, 3)	533 (4, 3)	1.22
recall-ru	2,419 (2, 3)	2,486 (2, 3)	1.03
recall-de	3,343 (4, 3)	2,783 (2, 3)	0.83
recall-en	7,155 (4, 1)	6,850 (4, 1)	0.96

Table 5. Replication of the corrected-protocol frontier. The binding level is 0.25 for recall-de and recall-en and 0.10 for the other cells, where the two levels coincide.

The preregistered protocol's frontier at $\hat{D} \leq 0.25$, for comparison with Table 2: translation-de 1,194 bits, translation-ru 699, recall-ru 3,101, recall-de 4,168, recall-en 24,686.

Bootstrap intervals over test examples for the gain of the frontier programs over the no-prefix model, evaluated at the mean prefix (per-example losses are stored for the mean only): 0.52 ± 0.01 bits per token on translation-ru, 0.50 ± 0.03 on translation-de and 5.69 ± 0.07 on recall-ru.

Repository `sirotenkogeorgii/prefix-programs`, branch `prefixlab`. `results_sync/` holds every result table, per-example losses, training histories and prefix checkpoints. `notebooks/01_colab_runner.ipynb` reproduces the runs on an A100 (about 36 hours); `notebooks/01_figures.ipynb` regenerates every figure and table from the result tables in under a minute; `notebooks/02_live_demo.ipynb` trains one prefix live.